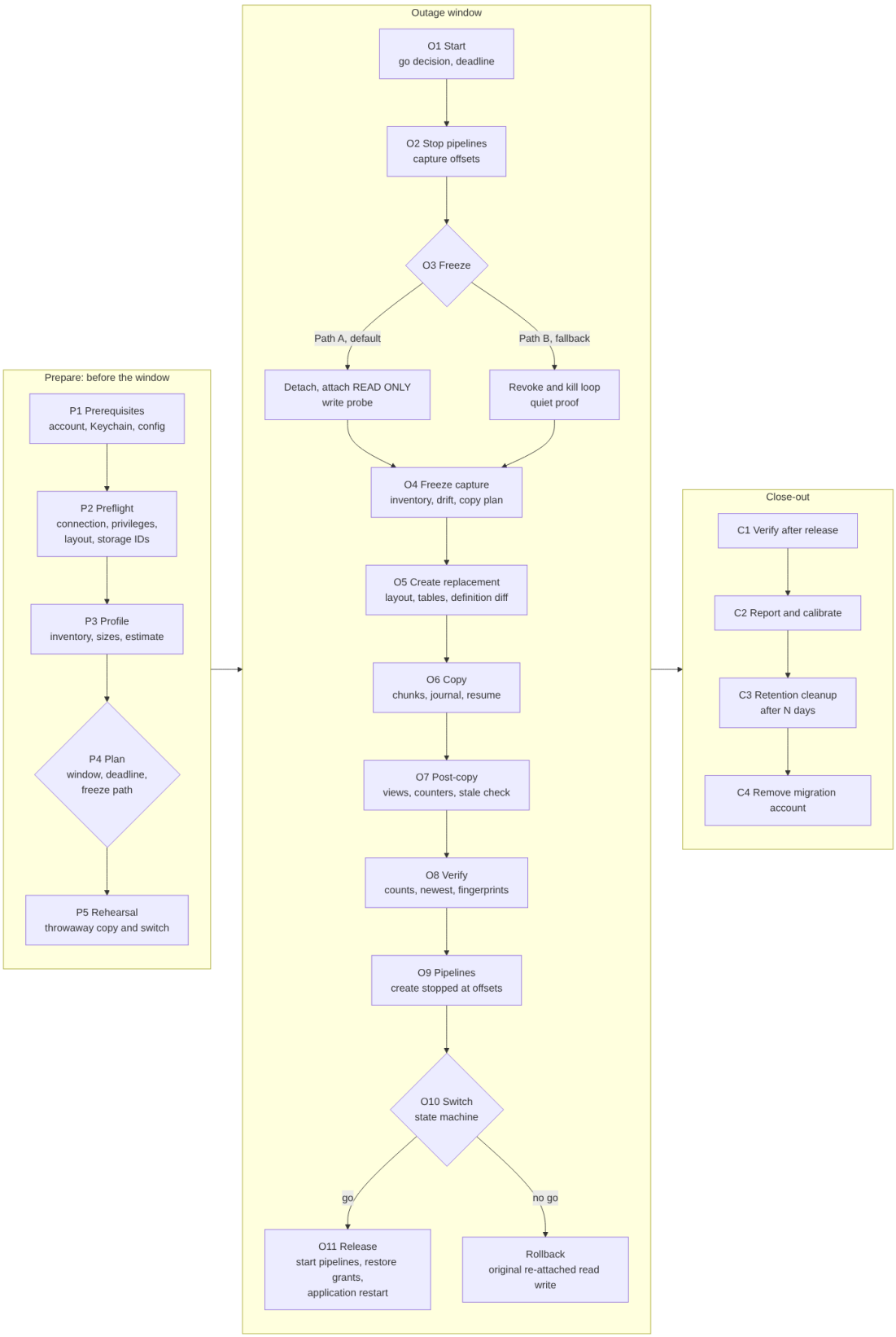
SingleStore Database Rebuild Kit

Step guide: the README and the step catalog

Author: John Bergamini

Version 1.0.0

Flow of the steps



SingleStore Database Rebuild Kit

Author: John Bergamini. Version 1.0.0. Proprietary; see LICENSE.

The kit rebuilds a SingleStore Helios database into a new partition layout and cuts it over under its original name inside one outage window. The first outage step makes the source read only. Everything after that is a copy, a verification and a switch, each behind a gate, each restartable from a terminal.

Who runs what

Identity	Who	Steps
runner	A SQL account the kit creates at P1 and drops at C4	P2, P3 tables, O4 to O8, C1, C2, watch
owner	The workspace admin SQL user, or the Portal identity in the owner notebook	P1 account, P3 pipelines, O2, O3, O7 views, O9, O10, O11, C3, C4, lockout
operator	The person running the kit	every command; records the gates a person decides
customer	The application owner	stops the application (if asked), hands over pipeline credentials, restarts the application at O11

The runner copies. The owner freezes, switches and releases. With an owner profile in the configuration the kit runs the admin steps itself; without one it prints each admin block and waits for `s2rebuild gate`.

Install

Python 3.10 or later on macOS or Linux. The Keychain password source is macOS only; elsewhere use `password_env`.

```
python3 -m venv .venv
.venv/bin/pip install -e .
.venv/bin/s2rebuild --help
```

Run the offline tests with `.venv/bin/pip install pytest` and `.venv/bin/python -m pytest`. No test opens a network connection.

Configure

1. Copy the example: `cp examples/config.example.yaml customer/config.yaml`.
2. Set the workspace host, the database names and the target layout. Every key is in docs/CONFIG.md.
3. Passwords: with `password_keychain` the kit fills the macOS Keychain items itself at P1 (`s2rebuild account-sql --execute` asks for the owner password when its item is missing, hidden, and generates the runner password to the workspace policy); or store them yourself (`security add-generic-password -a "$USER" -s s2rebuild-owner -w`), or name an environment variable with `password_env`. Pipeline credentials are asked for at P3 (or O9) and stored in the items named in `pipelines.credentials_keychain`; the variables in `pipelines.credentials_env` stay the fallback. The kit never prints, logs or writes a password or credential.
4. Names: `databases[].replacement` and `retained` may be left out. In `run.mode: production` (the default) O1 asks for them with the date default as the suggestion; `run.mode: test` (or `--mode test`) applies the defaults silently.
5. Everything a run writes stays under `customer/`. The rest of the folder is the static tree.

Steps

Each step prints what it did and the gate it needs. Exit codes: 0 done, 1 refused, 2 incomplete (rerun to resume), 3 verification mismatch, 4 gate pending or failed. The full catalog with inputs, outputs, statements and recovery is docs/STEPS.md.

Every command works on one run under `customer/runs/`. The P1 gate starts the run and prints its id. Export that id as `S2REBUILD_RUN`, so every later command names the same run. Without `S2REBUILD_RUN` or `--run ID`, a command uses the most recent run. `s2rebuild run` resumes that run and skips the steps that passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names the new run). Once a run exists, `s2rebuild run --run ID` refuses an id that names no run, so a mistyped id never starts another run. Every gate hint and every log header prints the run id with its `--run` form.

The sequence from P1 to the go and on to O2, where RUN_ID is the id the P1 gate printed:

```
s2rebuild account-sql
s2rebuild gate P1 --pass --note TEXT
export S2REBUILD_RUN=RUN_ID
s2rebuild check
s2rebuild profile
s2rebuild plan
s2rebuild gate P4 --pass --note TEXT
s2rebuild rehearse
s2rebuild run --from O1
s2rebuild gate O1 --pass --note TEXT
s2rebuild run
```

Prepare (before the window)

1. P1 Prerequisites. Print the runner account block with `s2rebuild account-sql` and have the admin run it, or run it with `s2rebuild account-sql --execute`: it asks for the owner password when its Keychain item is missing, prints the workspace password policy, generates a runner password that meets it, stores it in the runner's Keychain item and creates the account. Write the configuration. Gate: the account, the Keychain items and the configuration are in place; record it with `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`.
2. P2 Preflight. `s2rebuild check` connects as the runner (and the owner), checks every runner privilege by name, records the source's branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), confirms the replacement and retained names are free and the compute size fits the partition count. With `workspace.management_api_key_keychain` set it reads the workspace through the Management API: the state must be ACTIVE, and the autoscale and auto-suspend settings go onto the plan sheet; without a key it prints three questions (ACTIVE, autoscale off, auto-suspend off or a keep-alive job) for O1 to record. It refuses until P1 is recorded. Run it again any time: O1 needs P2 passed in the same run. Gate: every line ok.
3. P3 Profile. `s2rebuild profile` captures the inventory (tables, keys, sizes, accounts, sessions, views; with the owner profile the pipelines, the routines and the write-capable accounts, which the runner cannot see) and writes `profile.xlsx` with the estimate and, when a window is configured, the window fit. It refuses until P2 passed. Once P3 passed, a new capture needs `--redo`, which logs what it rewrites. `--dry-run` writes only `profile-dry-run.xlsx`, never the capture. Gate: the estimate fits the window, when one is configured. Routines (functions, table-valued functions, aggregates, procedures) are captured with their definitions and recreated at O5 and O7; `objects.routines.ignore` leaves one behind on purpose. A pipeline that loads into an ignored procedure refuses the capture, since O9 would

find no target. On route `kit` the kit then asks for the pipeline credentials of each source type (S3 key pair and token, Kafka SASL user and password, or key and value pairs for another type) and stores the CREDENTIALS JSON in the Keychain item named in `pipelines.credentials_keychain`; a blank answer skips the type, and the plan sheet and O9 list the pipelines that will not be recreated.

4. P4 Plan. `s2rebuild plan` prints the plan sheet: window and abort deadline (when configured), freeze path, step owners, pipeline credential route. Gate: the decisions are recorded with `s2rebuild gate P4 --pass --note TEXT`.
5. P5 Rehearsal. `s2rebuild rehearse` builds throwaway `s2rebuild_rehearsal_` databases, copies them with a SIGKILL test and a resume, verifies, freezes, switches and drops what it created. The killed chunk leaves a leaf lock for 15 to 20 minutes, so plan 30 minutes. Gate: P5 passed (the view at O7 and the switch need the owner profile).

Outage window

6. O1 Start. `s2rebuild run --from O1` checks that P2 passed in this run and that rehearsal evidence exists (P5 passed, or a passed rehearsal of any run), records the window and the abort deadline when a window is configured, records the configuration hash, asks for a replacement or retained name the configuration leaves to the kit (`run.mode: production`; `--mode test` applies the defaults), prints both names, and waits for the go decision: `s2rebuild gate O1 --pass --note TEXT`. Without a rehearsal, `--no-rehearsal NOTE` starts anyway and records the note. While P2 read autoscale on, O1 refuses until `--accept-autoscale NOTE` records the override; without an API key, `--workspace NOTE` records the answers to the three workspace questions as evidence of the go. The window is optional: without `window.start` and `window.end` there is no abort deadline, and the copy and the switch run without a time limit. Then `s2rebuild run` resumes the same run: it skips O1 and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.
7. O2 Stop pipelines. The owner stops every pipeline and the kit captures every definition and offset again after the stop. These are the offsets O9 replays. Gate: no pipeline Running; loaded and successful offsets agree on every partition; capture written.
8. O3 Freeze. Path A (default): DETACH, ATTACH READ ONLY, a write probe that must be refused. A DETACH that waits longer than `freeze.detach_wait_seconds` is cancelled and issued again with KILL. Path B (single database only): `s2rebuild lockout --hold` revokes write privileges and kills sessions until the quiet proof passes, then keeps passing until S1 of O10 has run; leave it running in a second terminal through O10. After a notebook or hand run, `s2rebuild step O3 --redo` runs the probe and records freeze_time. A run that stopped between DETACH and ATTACH resumes at the ATTACH: `s2rebuild step O3 --redo` sees the detached storage and the free source name and never detaches again. For a hand run, `s2rebuild owner-sql O3` writes that block to `owner-sql/O3-reentry.sql` beside `O3.sql`. The Portal may show 0 rows for the frozen source; the kit's counts are the reference. Gate: probe refused and READONLY (A), or quiet proof passed (B).
9. O4 Freeze capture. `s2rebuild run` captures the inventory after the freeze and reports drift against P3 (tables, columns, views, routines and pipelines; on Path A the pipelines are not listed on the READ ONLY source and the report says so). `s2rebuild profile --step O4` does the same by hand and stays for a rerun: once O4 passed, a new capture needs `profile --step O4 --redo`, which rewrites the copy plan. Both refuse until O1 to O3 passed. Gate: drift reviewed.
10. O5 Create replacement. The run continues (or `s2rebuild run --from O5`): the replacement is created with the target layout and every table definition is compared with the capture; its STORAGE_ID is recorded. Then the owner session creates the captured functions and table-valued functions, and the aggregates after the functions they name; the gate compares each definition with the capture. A routine body that names the source database refuses the step before any statement.

11. O6 Copy. Server-side INSERT ... SELECT in hash buckets, journaled, resumable. Stop it only with `s2rebuild pause` or the abort deadline: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes. Gate: every chunk done; per-table count proof all match.
12. O7 Post-copy. Views, then procedures, AUTO_INCREMENT counters and the stale check (counts and newest values) with re-copy of anything that moved. A completed re-copy marks O6 passed again, and the counters are set after the last re-copy. A view runs under its definer and C4 removes the runner, so the runner never creates one. With the owner profile the kit creates the views, keeping each original definer whose account still exists, then the procedures the same way (a procedure may read a view, and a pipeline INTO PROCEDURE at O9 needs its procedure). Without it O7 writes `owner-sql/O7.sql` and ends pending: run that block as the admin, then `s2rebuild step O7`. A view or procedure whose definition names the source database refuses the step. Gate: views and procedures created, zero mismatches.
13. O8 Verify. Counts, newest values and grouped fingerprints, tier 3 in parallel on `verify.sessions` connections, largest table first. A table over `verify.full_fingerprint_max_rows` (1 billion rows) is sampled as a contiguous range on its sort key or time column, the same predicate on both sides; `verify.fingerprint_budget_minutes` is an elapsed deadline after which a table is recorded not fingerprinted. Gate: tiers 1 and 2 pass for every table; tier 3 a full match everywhere, or a sampled or not fingerprinted result accepted with `s2rebuild step O8 --yes`, which accepts the recorded result without a rerun when no chunk was copied since. `step O8 --full TABLE` or `--fingerprint TABLE --percent N` reruns tier 3 for one table. A mismatch marks the buckets for re-copy; run `s2rebuild run --from O6` to copy them again and rerun O7 and O8.
14. O9 Pipelines. The kit reads each pipeline's credentials from its Keychain item (filled at P3), else from the variable exported for this step only; a source type still without credentials is asked for here, and a blank answer skips it. The kit recreates every other pipeline in the replacement, stopped, at the O2 offsets. Route customer: the customer runs the placeholder file and the kit checks. Gate: count (the skipped pipelines left out), none Running, offsets equal the O2 capture partition by partition.
15. O10 Switch. A state machine that ends with two entries: S1 detaches the source; S2 attaches the original storage read only under the retained name; S2b attaches the original record by its own syntax under the live name, proves it unchanged (the three smallest tables, no foreign session) and drops it; S3 attaches the replacement as a branch under the live name; S4 drops the replacement record; the gate lists both storages and expects exactly the live record and the retained record. A failed proof at S2b stops with the original attached read write under the live name, the rollback state. S0 refuses unless every planned table was verified after its last chunk was copied; without the owner profile O10 refuses on the same rule, and the printed block and the notebook cell start with a STOP while it holds. A re-copy rewrites `owner-sql/O10.sql` with a STOP; generate the blocks or the notebook again after O8 passes. On Path B, S0 and S1 refuse unless the last lockout pass was quiet within `freeze.quiet_max_age_seconds` (default 60); that refusal comes before the go, so a re-copy stays possible. The printed block and the notebook cell check instead, before the DETACH, that every privilege the lockout took is still revoked. Type the live name when asked (or pass `--yes`). Gate: the live name resolves to the new branch with the target layout and the capture counts. From S3 the live name accepts writes. A rollback (`s2rebuild owner-sql O10 --rollback` writes both legs) runs only while every table still holds the row count of the copy and O11 has not started; it detaches the live name only while it holds the recorded live record, then re-attaches the original record by its own ATTACH_SYNTAX before S2b ran (R2a), or after S2b attaches a read-write branch of the retained copy under the live name (R2b; R2c re-attaches the retained record itself when the server refuses a branch of a read-only attachment). No rollback step drops the retained copy. The count check cannot see an update that keeps the count, so a rollback after S3 is safe only when the application stayed stopped until O11, as the O1 note records.

16. O11 Release. Retention check per partition, grants restored, pipelines started; the customer restarts the application. Gate: application writing.

Close-out

17. C1 Verify after release. `s2rebuild step C1`: writer watch, count changes, pipelines Running with offsets advancing, application-managed pipelines back, no errors, grants compared. Gate: all ok and application writes seen. For a live database idle on purpose, pass `s2rebuild step C1 --idle-workload NOTE`. Without the owner profile the pipeline and grants checks are recorded as not checked.
18. C2 Report and calibrate. `s2rebuild step C2` writes `final.xlsx` and `run-report.md` and refits the throughput model into `customer/baseline.json`. With too few chunks to fit, the baseline in use stays, the reports say so, and C2 passes. When the kit cannot pass C1 or C2, record that step by hand with `s2rebuild gate ID --pass --note TEXT --override NOTE`.
19. C3 End state. `s2rebuild step C3` checks that the workspace holds exactly two entries for the run, the live database and the retained copy (see below), that the live name resolves to the recorded live record, and that a write on the retained name is refused; the listing goes into the journal and the final report. It issues no DETACH, ATTACH or DROP. When the gate finds a third entry and the journal shows no S2b or S4 (a switch made by an older kit), the step prints the repair block, says that the live name is detached for about 30 seconds, and runs it after you type proceed (or with `--yes`): detach the live name, attach the original record by its own syntax, check it, drop it, re-attach the live record by its own syntax, drop the replacement record, then the gate again. Without the owner profile the block is printed and the gate waits for a person.
20. C4 Remove the migration account. `s2rebuild step C4` drops the runner account; delete its Keychain item. C3 must be passed. The step stops before DROP USER while a view or a routine in the live database has the runner as its definer.
21. C5 Delete the retained original, later and by the customer. `s2rebuild cleanup` lists what the run left: the live record, the retained copy with its record, its size in GiB and the retain-until date. After that date `s2rebuild cleanup --delete <retained>` prints the record and asks you to type the name again; anything else keeps the copy and records "kept by customer". The delete is DETACH, re-attach by the record's own syntax read write, a record check, DROP, each behind its check, then a check that the live name still resolves to the live record. `--yes` does not apply, and no configuration key makes a run delete the retained copy.

After a run the workspace holds two databases: the live database under the live name, a branch of the replacement with the target layout, and the retained copy of the original under the retained name, attached read only on the original storage. The switch (O10) ends there on purpose: the original's own record and the replacement are dropped inside the switch (S2b and S4), after the proof that the copy equals the original and that the live branch is independent of the replacement, so no second outage is needed and nothing is left to delete by hand. The retained copy is the way back and the audit copy; the kit never deletes it. C2 records `retain_until_utc` (the switch time plus `closeout.retain_days`, 90 by default) in the journal, the final workbook and `run-report.md`, with the C5 command.

Troubleshooting

- Re-copy after O8: `s2rebuild mark-recopy TABLE [--bucket N]` sets O6, O7 and O8 back to pending and clears the table's verification. Run `s2rebuild run --from O6`; O6 copies the buckets again, then O7 and O8 run again. The switch refuses until O8 passes again. A fresh-table load reopens O7 and O8 the same way. Both rewrite `owner-sql/O10.sql` and `O10-rollback.sql` with a STOP;

after O8 passes, run `s2rebuild owner-sql O10` (and `--rollback`) or `s2rebuild owner-notebook` again, and never run the O10 cells of an older notebook. Once the switch may have begun (O10 passed, the go recorded, an O10 gate, or the retained or live attachment recorded), a re-copy is refused; a new copy is a new run.

- Autoscale or a restart during the copy (error 1997, "transaction was already in its commit phase"): every chunk in flight fails when a node is replaced. The chunk is reconciled and requeued without consuming an attempt. After `copy.max_consecutive_failures` (default 6) failures in a row the circuit breaker stops new chunks and O6 ends INCOMPLETE with "chunks are failing in a row; nodes may be restarting, see MV_EVENTS; rerun with --retry-failed when the workspace is steady". Check MV_EVENTS, turn autoscale off for the window, then `s2rebuild run --from O6 --retry-failed`. P2 reads the setting through the Management API when `workspace.management_api_key_keychain` is set and O1 refuses while it is on.
- A node reset the connection (driver error 2003, 2004, 2006 or 2013): the command prints one plain line, "a node reset the connection; the workspace may be scaling or restarting; rerun the step", and exits 2; the raw error is in the run's log file. The step is recorded failed and re-enters from the journal on the rerun. A workspace that suspended during the window shows the same: resume it in the Portal, run `s2rebuild check`, then the step again.
- Auto-suspend: while a step runs, the kit's keep-alive thread sends `SELECT 1` every `workspace.keepalive_minutes` (default 15; 0 disables) on its own runner connection. Between commands nothing runs, so disable auto-suspend for the window, or run a notebook job with the same query (STEPS P4).
- Lock waits (error 1205): the kit serializes the table, probes the lock and retries after `copy.lock_retry_seconds`. A lock held by a killed client clears in 15 to 20 minutes. A chunk fails after `copy.max_lock_waits` lock waits and O6 stops INCOMPLETE with the command to run: `s2rebuild step O6 --fresh-table TABLE` loads the table into a new table and renames it. It refuses past the abort deadline or while the stop file exists. After a failure, rerun the same command; it resumes and does not load a complete fresh table again.
- Detach waits: a DETACH blocked by writers is cancelled after `freeze.detach_wait_seconds` and issued with KILL when `freeze.detach_kill` is true. With `detach_kill: false` the step stops; run `s2rebuild lockout` as a bridge, then `s2rebuild step O3 --redo`.
- Grants restored by another session: the lockout logs `GRANT RESTORED BY ANOTHER SESSION` and revokes again on the next pass, with `--hold` also after the quiet proof. Find the job that restores them before the night.
- Name collisions: names are not unique on Helios, and a STORAGE_ID names a storage that several branch records share. Every ATTACH names a STORAGE_ID, and every check of the switch, the rollback, C3 and abandon compares the branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID) with the one the journal recorded. `s2rebuild check` refuses a replacement or retained name that already exists.
- Pipelines the application recreates: list them in `databases[].app_managed_pipelines`. They are stopped at O2, never replayed, and C1 reports whether they came back.
- Configuration changed after O1: O1 records the configuration hash. Every command that opens the run journal refuses a configuration that hashes differently, until `--accept-config-change NOTE` records both hashes and the note in the journal. A changed database name is refused with or without it: use the configuration of the run. `s2rebuild status` reports both.
- Abandon a run before the switch: run `s2rebuild pause` and wait until the copy step ends, so no chunk still reads the source. `s2rebuild abandon` prints the block; `s2rebuild abandon --execute`

`--note` TEXT runs it with the owner profile. It attaches the source READWRITE again as its recorded record, drops the replacement only while its name holds the record O5 recorded and no branch was made from it, and closes the run: `run`, `step` and `gate` refuse it afterwards. Then remove the runner with `s2rebuild account-sql --drop --execute`. A new run starts at P1. Once the switch may have begun, abandon refuses; the way back is `s2rebuild owner-sql O10 --rollback`.

- An admin step stopped at a check: the error names the check, the expected value and the observed value. Fix the cause and rerun the step; the freeze and the switch re-enter from the live attachment state.

Documents

- docs/STEPS.md: every step in full.
- docs/CONFIG.md: every key, flag, variable and exit code.
- docs/COVERAGE.md: the coverage grid, what the kit migrates and what it does not.
- docs/CODE-GUIDE.md: modules, data flow, decisions you can change.
- docs/LESSONS.md: Helios facts and run lessons.
- docs/ARCHITECTURE.md: the design.
- `docs/s2rebuild-guide.pdf`: this README and STEPS with the flow diagram, built by `docs/build_pdf.py`.
- `tests/engine/helios-<version>.json`: the record of the engine the test fake is built from, written by `s2rebuild engine-capture --fixture-database NAME` with the owner profile (reads only); refresh it per engine version as docs/CODE-GUIDE.md says.

Steps

Author: John Bergamini

One section per step. Each lists the purpose, the identity, inputs, outputs, the statements the kit issues, the gate, how to re-enter and what to do when it fails. Admin steps print their block (also written to `customer/runs/<run>/<db>/owner-sql/<step>.sql`, mode 0600); with the owner profile the kit executes it statement by statement and stops at the first check that does not return the expected value. Every statement passes the step's guard first.

Commands below assume `s2rebuild` on the path and `customer/config.yaml` as the configuration. Add `--db NAME` when several databases are configured.

A step that passed is skipped on a rerun only while every step before it is passed. When an earlier step is reopened (a re-copy reopens O6, O7 and O8), the passed step is refused like a step that never ran. The refusal names the steps to pass first, earliest first, and how: `s2rebuild gate ID --pass` for a step a person records, `s2rebuild run --from ID` for a step the kit runs. Follow them in that order, since `run --from` stops at a gate a person has not recorded.

The same rules hold for every command that runs a step: `run`, `step`, `check` (P2) and `profile` (P3, O4).

1. A step whose earlier steps are not passed is refused for every selected database before the command connects or writes. Gate: the refusal names the steps to pass. A dry run reports it and goes on.
2. `profile` captures a passed step again only with `--redo`, and the log names what it rewrites. `check` reruns a passed P2 without it.
3. `gate` records a step whose earlier steps are not passed, with a warning that names them. Gate: the warning is printed and logged.
4. O1 records the database names and the configuration hash. Every command that opens the run journal refuses changed names. It refuses a changed configuration unless `--accept-config-change NOTE` records an event with both hashes and the note. Gate: `s2rebuild status` shows no warning. A dry run and `status` report a changed hash and go on.
5. Every command works on the run that `--run ID` or `S2REBUILD_RUN` names, else on the most recent run. `run` resumes that run and skips the steps that passed. It starts a run only when none exists, or with `--new` (`--new --run ID` names it; an id that exists is refused). Once a run exists, `run --run ID` refuses an id that names no run. Every gate hint and every log header prints the run id with its `--run` form. Gate: one run directory holds P1 to O11 of the night.

P1 Prerequisites

- Purpose: an account for the copy, its password stored, the configuration written.
- Identity: owner (account) and the operator.
- Inputs: `examples/config.example.yaml`, the admin.
- Outputs: the runner account with the privileges of section 5 of the architecture; a Keychain item or variable per identity; `customer/config.yaml`.
- Statements: `s2rebuild account-sql` prints CREATE USER and GRANT with checks before and after; `--execute` runs them with the owner profile and injects the password at execution only.
- Owner password: with `connections.owner.password_keychain`, `account-sql --execute` asks for the owner password (hidden, never echoed) when the item is missing and stores it with `security add-generic-password -U`; with `password_env` the variable is read as it is.

- Runner password: with `connections.runner.password_keychain`, `account-sql --execute` reads the `PASSWORD_` variables through the owner session, prints the policy, generates a 32 character password that meets it (every character class, no character twice in a row, no three sequential characters; the policy minimum when it asks for more) and stores it in the item before CREATE USER. Nothing is asked and the value is never printed. An account that exists already keeps its item, and the block stops at its first check. With `password_env` the variable is used as it is.
- Gate: the runner account, the Keychain items (or variables) and the configuration are in place. Record: `s2rebuild gate P1 --pass --note TEXT`. When no run exists yet, this gate starts the run and prints its id; export it as `S2REBUILD_RUN`. Once a run exists, `--run ID` must name it.
- Re-enter: rerun `account-sql`; its first check stops when the account already exists.
- When it fails: a missing privilege is named by `check` (P2); grant it and rerun `check`.

P2 Preflight

- Purpose: prove the connection, privileges, layout, storage id, names and compute size before anything else.
- Identity: runner, plus the owner when configured.
- Inputs: the configuration; P1 recorded in the run. Until then `check` refuses and creates nothing.
- Outputs: the source STORAGE_ID in the journal, the engine version, a preflight record.
- Statements: SELECT, USE and SHOW only: CURRENT_USER and version, USER_PRIVILEGES for the runner, DISTRIBUTED_DATABASES, MV_CLOUD_ATTACHED_DATABASES, MV_NODES leaf cores; with the owner profile the PIPELINES count, ROUTINES, SHOW AGGREGATES and SHOW CREATE per routine. With `workspace.management_api_key_keychain` set, one Management API read, `GET /v1/workspaces/<workspace.id>` with the key from the Keychain as a bearer token (the key never prints).
- Workspace: with the API key, the state must be ACTIVE (a suspended, resuming or scaling workspace fails the line; resume it or wait, then run `check` again), and the autoscale and auto-suspend settings are recorded and printed on the plan sheet; O1 refuses while autoscale is on. Without the key P2 prints three questions, "is the workspace ACTIVE?", "is autoscale off for the window?", "is auto-suspend off for the window, or a keep-alive job in place between the commands?", and O1 records the answer from `--workspace NOTE`. The API field names are a best guess until the live ladder confirms them (LESSONS fact 41); a field the API does not answer prints "not reported".
- Gate: every line ok: all runner privileges present, source reported with a STORAGE_ID, replacement and retained names free, leaf cores times 8 at or above the target partitions (when MV_NODES is readable; otherwise confirm in the Portal), workspace ACTIVE when the API is read. The routine line prints "N routines (f functions, t TVFs, a aggregates, p procedures), M ignored by configuration" with the owner profile; without it, "routines: not visible to the runner; owner profile needed", never "none", since the runner sees no routine.
- Re-enter: `s2rebuild check` any time; a passed P2 runs again without `--redo` and records its result again. O1 needs P2 passed in the same run.
- When it fails: each failed line says what to change (grant, rename, compute size).

P3 Profile

- Purpose: know the database: tables, keys, sizes, accounts, sessions, views, routines, pipelines; estimate the copy.

- Identity: runner; owner for the pipelines, the routines and the write-capable accounts (the runner sees no routine and no other account's grants).
- Inputs: the configuration, `customer/baseline.json` when present; P1 and P2 passed in the run.
- Outputs: `inventory/P3/inventory.json`, `tables.csv` (with `storage_type`, `reference` and `copy_path` per table), `definitions/` with `definitions/routines/<name>.<kind>.sql` per routine (kind function, tvf, aggregate or procedure), `pipelines/` (0600, CREDENTIALS replaced by `<<CREDENTIALS>>`), `reports/profile.xlsx` with the estimate, bands and window fit.
- Statements: SELECT, USE and SHOW only (SHOW CREATE PIPELINE ... EXTENDED, ROUTINES, SHOW AGGREGATES and SHOW CREATE FUNCTION, AGGREGATE or PROCEDURE with the owner profile). A reference table (`CREATE [ROWSTORE] REFERENCE TABLE`) is always counted with `SELECT COUNT(*)`, because `TABLE_STATISTICS` reports no rows for it; a count that returns no row refuses the capture by table name. Its plan is one bucket on the staging path and the estimate counts its rows at the insert rate twice (staging table, then the reference table); the table tab says "reference table, copied through a staging table".
- Pipeline credentials: on route `kit`, when the capture holds pipelines with a CREDENTIALS clause, the kit asks for the credentials of each source type that has none yet, by field: S3 access key ID (shown), secret access key (hidden), session token (hidden, Return to skip); Kafka SASL user (shown) and password (hidden); for another source type it prints the CONFIG keys the pipelines use and asks for credential keys and values until a blank key. It assembles the CREDENTIALS JSON and stores it in the Keychain item named in `pipelines.credentials_keychain` for that type (a type without an item is not asked; export its `pipelines.credentials_env` variable at O9 instead). A blank first answer skips the source type: the journal records it (`pipeline_skip`), the plan sheet and the O9 block list the pipelines that will not be recreated, the O9 count gate leaves them out, and the final report names them with the placeholder file. No prompt without pipelines, on route `customer`, or without a terminal (O9 asks again). Nothing typed is printed, logged or written to a file.
- Gate: no pipeline loads into a procedure that `objects.routines.ignore` lists or the capture lacks (O9 would find no target); with a window configured, the estimate at the high band fits before the abort deadline. The plan sheet and the workbook Summary print the routines by kind and the ones ignored by configuration; without the owner profile they print "not captured".
- Re-enter: once P3 passed, `s2rebuild profile --redo` captures again; the log names what it rewrites. `--exact-counts` for COUNT(*) per table. `--dry-run` writes only `reports/profile-dry-run.xlsx` (or `--out`), never the capture under `inventory/`.
- A stop part way (a lost connection, Ctrl-C) leaves P3 not passed; a plain `s2rebuild profile` captures again.
- When it fails: `DOES NOT FIT` exits 4; widen the window, raise workers, or split the databases across nights (`--all-databases` ranks them). `P3 refused` exits 1, records P3 failed and writes no workbook; the refused capture stays under `inventory/P3/` for review and the plan sheet. It names each pipeline whose load target is a procedure the replacement will not hold; remove the name from `objects.routines.ignore` (or list the pipeline as application managed) and run `s2rebuild profile` again. A pipeline with an empty definition, or a CREDENTIALS clause the kit cannot replace, refuses the capture by name and nothing is written; run `s2rebuild profile` again.

P4 Plan

- Purpose: the decisions people make before the night.
- Identity: the people named in the plan.

- Inputs: `s2rebuild plan`.
- Outputs: a recorded gate.
- Statements: none.
- Gate: freeze path, window and abort deadline (when configured), the named owner for "application stopped", the pipeline credential route and handover, the retained name and retention days, recorded in the configuration; autoscale off for the window; the auto-suspend choice below.
- Auto-suspend between commands: while a step runs, the kit's keep-alive thread sends `SELECT 1` on its own runner connection every `workspace.keepalive_minutes` (default 15; 0 disables), so a long statement or a wait inside a step never looks idle. Between two commands nothing runs (the wait for a gate, the time between O8 and O10). Choose one for the window and record it in the P4 note: disable auto-suspend for the window in the Portal (the default choice), or a notebook job of the customer that runs `SELECT 1` every 15 minutes through the window. The plan sheet prints the workspace state and settings P2 read, or the three questions when no API key is configured.
- Re-enter: edit the configuration and print the plan again.

P5 Rehearsal

- Purpose: run the exact steps on throwaway databases first, including a hard kill and the switch.
- Identity: runner and owner.
- Inputs: `--rows` (default 1000000), `--prefix` (must start with `s2rebuild_rehearsal_`).
- Outputs: `customer/runs/<run>/rehearsal/timings.json` with every step time, the interrupted and reconciled chunks and the Portal cleanup note.
- Statements: CREATE and DROP DATABASE on the prefixed names only, the copy, verify and switch statements on them.
- Gate: P5 passed: copy, SIGKILL and resume (exactly the killed chunk reconciled), the view created by the owner at O7, verify, freeze and switch completed. Without the owner profile O7 ends pending, the view and the switch are not rehearsed, and P5 stays pending.
- Re-enter: `s2rebuild rehearse --cleanup --prefix PREFIX --yes`, then rehearse again.
- When it fails: the kit drops only the databases it created in that run.

O1 Start

- Purpose: start the window with a recorded go decision.
- Identity: runner and the operator.
- Inputs: the window in the configuration, when there is one; P2 passed in this run; rehearsal evidence: P5 passed in this run, or a rehearsal of any run under `customer/runs/` that records P5 passed.
- Outputs: names, window, abort deadline (end minus restart allowance, buffer and the longest modeled chunk) and the configuration hash in the journal. Without a window O1 records an empty window and deadline, which also clears any deadline an earlier O1 recorded; the copy, the switch and the lockout then have no time limit.
- Names: `run.mode` is `production` (the default) or `test`; `--mode` overrides it. In production, when `databases[].replacement` or `retained` is not set in the configuration, O1 asks for each name after its checks passed, with the kit's date default as the suggestion (Return keeps it), checks that it is an identifier unused by the run, records the answers in the journal (a `names` event and the O1 note) and prints both names before the go. Every later command reads the typed names back from the journal. In

test mode the defaults apply silently. Without a terminal, production mode refuses and says to set the names, type them at a terminal, or run with `--mode test`.

- Several databases: `s2rebuild run` runs O1 for every selected database, then stops once for the go. `s2rebuild gate O1 --pass` records the go in every journal. A refused or failed O1 stops the run at once.
- Statements: none beyond reading the capture.
- Refusal: without P2 passed in this run, or without rehearsal evidence, O1 refuses before it records the window, the names or the configuration hash; its note says why. Run `s2rebuild check` in this run, or `s2rebuild rehearse`. To start without a rehearsal, run `s2rebuild run --from O1 --no-rehearsal NOTE`; the note is recorded as an event and in the O1 note.
- Workspace: O1 prints the state, autoscale and auto-suspend settings P2 read through the Management API. While autoscale is on, O1 refuses before any journal write: turn it off for the window and run `s2rebuild check` again, or override with `s2rebuild run --from O1 --accept-autoscale NOTE`, recorded as an event and in the O1 note. Without an API key O1 prints the three questions of P2 and records the operator's answer from `s2rebuild run --from O1 --workspace NOTE` as evidence of the go (an event and the O1 note); without the flag the answer line says none is recorded.
- Gate: deadline in the future when a window is configured, P2 passed in this run, rehearsal evidence present or waived, go recorded: `s2rebuild gate O1 --pass --note TEXT`. The note records whether the application stays stopped until O11. Only a stopped application keeps the O10 rollback possible after S3.
- From O1 on: a changed configuration is refused by every command until `--accept-config-change NOTE` records it; a changed database name is refused with or without it.
- A go recorded in a journal where O1 never ran is recorded with a warning: the checks and records of O1 are missing there. Run `s2rebuild run --from O1 --to O1 --redo`, then record the go again.
- Re-enter: `s2rebuild run` resumes the same run. It skips O1 once the go is recorded and continues at O2. `s2rebuild run --new` starts another run instead, which needs its own P1 and P2.

O2 Stop pipelines

- Purpose: stop every pipeline and record the final bookmark of each.
- Identity: owner.
- What a pipeline and an offset are: a pipeline is a server-side ingestion job reading a source (a Kafka topic, an S3 prefix and others) into one table. Its offset is its bookmark per source partition. A pipeline recreated with its recorded offsets resumes where the old one stopped: no message skipped, none loaded twice.
- Outputs: `inventory/O2/` with every definition (credentials replaced by the placeholder) and the offsets read after the stop; journal pipeline rows.
- Statements: STOP PIPELINE per pipeline (skipped only for one already Stopped, so a pipeline in state Error is stopped too), a check that none is Running, then the capture. The stop list is the earlier capture plus any pipeline the live view shows.
- Gate: no pipeline Running; every captured pipeline Stopped; LATEST_LOADED_OFFSET equals LATEST_SUCCESSFUL_OFFSET on every partition, since O9 replays the committed position. A pipeline that runs again before the capture refuses the gate. A partition where the two offsets differ refuses it too, a NULL successful offset beside a loaded one included, with the partitions listed; rerun O2 once they agree. A pipeline whose SHOW CREATE PIPELINE returns no CREATE statement, or whose CREDENTIALS clause has a form the kit cannot replace, refuses the capture by name and nothing is stored; rerun O2. A topic, table or column named credentials does not refuse it.

- Notebook route: the O2 cell's result carries the capture; `s2rebuild gate O2 --pass --note TEXT --evidence FILE` imports it.

O3 Freeze

- Purpose: make the source read only for every account.
- Identity: owner; the write probe runs as the runner.
- Path A (default): checks (READWRITE, recorded STORAGE_ID, no pipeline Running), DETACH, check detached, ATTACH ... READ ONLY STORAGEID ..., checks (READONLY, same STORAGE_ID), then the write probe: a DELETE for a value that cannot exist on a string column (or an integer key below its minimum). Pass means the server refused it as read only. Then freeze_time and, with the owner, a grants reference for C1.
- DETACH waits: after `freeze.detach_wait_seconds` the kit cancels the waiting DETACH on its own connection and issues DETACH ... KILL when `freeze.detach_kill` is true; otherwise it stops and the lockout serves as a bridge.
- Path B: `s2rebuild lockout`: snapshot every write-capable account at every scope (global, database, table) to the journal and `grants-snapshot.json` (0600), revoke exactly those privileges, hold connections, and on every pass re-revoke grants restored by another session and kill every session of those accounts (idle ones too) from the connection that sees it. After the configured quiet passes the quiet proof runs; it records freeze_time when pipelines are stopped, no foreign session is active and counts and newest values hold for a minute.
- Path B is single database: with more than one configured database, `check`, `run`, `step` O3, O10 and O11, `lockout` (with or without `--hold`), `owner-sql` O3, O10 and O11, `owner-notebook`, and `gate` O3, O10 and O11 `--pass` refuse. They write no block, record no gate, take no snapshot and revoke nothing.
- Path B hold: start it as `s2rebuild lockout --hold` in a second terminal and leave it running through O10. After the quiet proof it keeps passing (re-revoke, kill, one line per pass) and writes each pass to the journal. It stops by itself once S1 of O10 has run, or on `--until`, the stop file or Ctrl-C. O10 refuses at S0, before the go, while the last pass is missing, older than `freeze.quiet_max_age_seconds` or not quiet. The hold needs the owner profile; on the hand and notebook routes the O10 forward block checks the privileges live instead (O10 Switch, S1).
- Gate: Path A, probe refused and READONLY; Path B, quiet proof passed.
- After the probe the log says that the Portal may show 0 rows for the tables of the frozen source (fact 38); the kit's counts in the journal and the O4 capture are the reference. The step line reads `[O3] Freeze (admin; owner)`: the redactor masks the owner and runner user names only where they stand as a user, never in a label.
- Re-enter: `s2rebuild step O3 --redo`. When the source is already READONLY on the recorded STORAGE_ID the kit only checks, probes and records freeze_time, with or without the owner profile.
- Re-enter between DETACH and ATTACH: when no database carries the source name and the recorded STORAGE_ID shows IS_DETACHED = 1, `s2rebuild step O3 --redo` prints the re-entry block to `owner-sql/O3-reentry.sql` and runs it with the owner profile: checks (storage detached, no database named source or replacement), ATTACH ... STORAGEID ... READ ONLY, the READONLY checks, then the probe and freeze_time. It never detaches again. The kit reads this position with the owner session when the owner profile is configured, otherwise as the runner. Without the owner profile, run the printed block, then `s2rebuild step O3 --redo`. The notebook O3 cell runs the same block (`O3-reentry`) when the whole block stops at its first check with no rows. A database under the source

name on another STORAGE_ID still refuses at the first checks.

- Hand route: `s2rebuild owner-sql O3` writes and prints both blocks, `owner-sql/O3.sql` and `owner-sql/O3-reentry.sql`. The whole block opens with a note that says when to run the re-entry block instead: its first check finds no rows. Gate: the re-entry block's checks pass; a database under the source name stops it before the ATTACH.

O4 Freeze capture

- Purpose: the inventory the copy uses, taken after the freeze.
- Identity: runner.
- Inputs: `s2rebuild run` takes the capture itself after the O3 gate (the runner session reads; the owner session, when configured, reads the routines and the write-capable accounts); `s2rebuild profile --step O4` stays for a rerun (`--redo`), for `--exact-counts` and for `--all-databases`. Both refuse until O1 to O3 passed, before they connect or write. A capture already present is checked and kept, never taken again without `--redo`.
- Outputs: `inventory/O4/`, `drift.json` against P3, the copy plan. Reference tables are counted exactly again, as at P3, so the plan carries the frozen count.
- Gate: drift reviewed (a routine added, removed or changed since P3 is listed by kind and name; a pipeline added, removed or whose definition changed is listed by name; states and offsets are not drift); the load-target rule of P3 holds again, so a pipeline into an ignored procedure refuses O4 and records it failed; O5 and later refuse until O4 passes.
- Pipelines on Path A: the source is READ ONLY, where `PIPELINES` is empty (fact 23). The capture reads no pipeline view and records "pipelines not listed: PIPELINES is empty on a READ ONLY attachment; O9 replays the O2 capture taken after the stop"; the drift report prints that note once instead of every pipeline removed. On Path B the pipelines are listed and compared.
- Re-enter: once O4 passed, `s2rebuild profile --step O4 --redo` captures again and rewrites the copy plan; the log says so. O5 and O6 refuse a plan that differs from the journal. A stop part way leaves O4 not passed; a plain rerun captures again. A dry run writes nothing under `inventory/`.

O5 Create replacement

- Purpose: the replacement database with the target layout and identical table definitions, then the functions, table-valued functions and aggregates of the capture.
- Identity: runner for the database and the tables; owner for the routines.
- Statements: CREATE DATABASE ... PARTITIONS ... SUB_PARTITIONS, CREATE TABLE ... LIKE per table, SHOW CREATE TABLE (runner); USE, CREATE FUNCTION and CREATE AGGREGATE naming the replacement, with the ROUTINES and SHOW AGGREGATES reads and SHOW CREATE per routine (owner).
- Routines: after the tables, the owner session creates the functions and TVFs, then the aggregates (an aggregate names its functions), each skipped when it exists and checked after; a routine that fails is tried again after the others, in rounds, and a round that creates none refuses the step with each routine and its error number. The DEFINER rule of the views applies: the original definer is kept while that account exists and is not the runner, else the owner; on error 1227 the routine is created again without the clause and the reason is recorded per routine. A routine body that names the source database refuses the step before any statement. A routine `objects.routines.ignore` lists is left behind and named in the log.
- Outputs: the replacement STORAGE_ID in the journal (the switch attaches it by that id); one `routine` event per routine with the definer choice.

- Gate: layout and durability verified; every table definition equals the capture (only the AUTO_INCREMENT counter may differ); every expected function, TVF and aggregate exists in the replacement with the captured definition (normalized, DEFINER aside).
- When it fails: `O5 stopped` names each routine still missing with its error; fix the cause (a function an aggregate names is ignored by configuration, say) and rerun O5; routines already created are skipped.

O6 Copy

- Purpose: copy every row.
- Identity: runner.
- Statements: INSERT INTO replacement SELECT FROM source WHERE CRC32(key) MOD B = k per bucket; reconciliation DELETE of a started or failed bucket before it is re-inserted.
- Behaviour: a reference table (marked `reference` in the capture, one bucket, `copy_path` staging) copies through a sharded staging table in the replacement (`<t>__stage`, created from its definition without keys, filled from the source, copied into the reference table, dropped), because an INSERT into a reference table cannot read a READ ONLY source; its count proof compares the exact count the capture took; a queue per table; above 8 workers no table takes more than half of them (`copy.table_concurrency: auto`); a table that hits a lock wait (1205) is serialized and probed; the journal is written before each statement; the abort deadline and `s2rebuild pause` stop new chunks while running ones finish. A chunk fails at its `copy.max_lock_waits`-th lock wait, and the O6 note names `s2rebuild step O6 --fresh-table TABLE`. Ctrl-C stops new chunks; a second Ctrl-C only prints that a hard stop needs kill, and the workers still finish their chunks. Never hard-stop the copy: a killed client leaves a prepared transaction holding a leaf table lock for 15 to 20 minutes.
- Transients and the breaker: error 1997 ("transaction was already in its commit phase") means a node was replaced under the chunk (autoscale, LESSONS fact 40); the chunk is marked failed, so its rows are reconciled, and goes to the back of its table's queue without consuming an attempt (journal event `transient`). After `copy.max_consecutive_failures` (default 6) chunk failures with no success between them (1997 counted, lock waits not), the circuit breaker stops new chunks, the running ones finish, and O6 ends INCOMPLETE (exit 2) with "chunks are failing in a row; nodes may be restarting, see MV_EVENTS; rerun with --retry-failed when the workspace is steady" (journal event `breaker`). A connection reset of the step itself (driver error 2003, 2004, 2006 or 2013) ends the command with one plain line, "a node reset the connection; the workspace may be scaling or restarting; rerun the step", exit 2, the raw error in the log file.
- Gate: every chunk done; per-table count proof all match.
- Re-enter: `s2rebuild run --from O6` resumes; `--retry-failed` reruns parked chunks; `s2rebuild step O6 --fresh-table TABLE` loads a locked table into a new table and renames it. The fresh-table load refuses past the abort deadline, while the stop file exists, and when the fresh name would be over 64 characters or a planned table. It reuses only a fresh table its journal recorded; another table under that name refuses, and a missing replacement table is never recreated from the source. After a failure, rerun the same command: it resumes from the recorded fresh table and does not load it again once it is complete. After a re-copy mark, or a copy run that finished the table, it loads the table again. The fresh-table load replaces the whole table, so before its first statement it clears the table's verification and sets O7 and O8 back to pending when they passed, and the O10 block files get a STOP as for a re-copy. After it, run `s2rebuild run --from O6`: O7 sets the counters of the new table, O8 verifies it.
- Re-copy: `s2rebuild mark-recopy TABLE [--bucket N]` marks buckets for the next copy run, which deletes and re-inserts them. In the same journal write it sets O6, O7 and O8 back to pending when they passed, and clears the table's `verified_at_utc` and `target_count`. With several databases it

checks every selected journal first, so a refusal for one marks none. Just before that journal write it rewrites `owner-sql/O10.sql` and `owner-sql/O10-rollback.sql`, when they exist, so each starts with a STOP, and it says that a notebook generated before must not run its O10 cells. An O7 or O8 re-copy mark and a fresh-table load do the same. Then:

1. Run `s2rebuild run --from O6`. O6 copies the marked buckets again. Gate: O6 passed.
2. O7 runs again: counters after the re-copy, then the stale check. Gate: O7 passed.
3. O8 runs again on the re-copied rows. Gate: O8 passed. Until then S0 refuses the switch and the rollback block stops.
4. Before a hand or notebook switch, generate the O10 blocks again: `s2rebuild owner-sql O10` and `s2rebuild owner-sql O10 --rollback`, or `s2rebuild owner-notebook`, and use only the new notebook. Gate: the forward block holds no STOP.

- Re-copy after the switch began: refused. Once O10 passed, the go decision is recorded, an O10 gate is recorded (pass or fail), a refused `gate O10 --pass` recorded the switch its evidence shows, or the switch time or the retained or live attachment is recorded, `mark-recopy` and O6, O7 and O8 refuse with nothing written. A passed O6 to O8 is still skipped, so `s2rebuild run` continues to O11. A new copy is a new run.
- An O10 that stopped before the go decision issued nothing, so it does not block a re-copy: a Ctrl-C at the typed-name prompt, a lost owner connection, a stop during S0. A refusal at S0 or at an S1 before-check does not block it either: those checks run before the go, the Path B lockout pass among them. When O10 reads a switch already under way (a hand S1, say), it records that before the go, and the re-copy stays refused. After a hand or notebook switch, record it with `s2rebuild gate O10 --pass` or `--fail`, so the journal knows.
- While O10 runs, it holds the copy lock of the journal. `mark-recopy`, O6 and an O7 re-copy round in another terminal refuse until O10 ends, with nothing marked or copied; the refusal names the process. As O10 records the go decision, it reads the journal part of S0 again in the same journal write. A re-copy mark (an O8 mismatch, say) or O6 to O8 started in another terminal while the operator typed the live name refuses the go, with nothing issued. Then run `s2rebuild run --from O6`, and O10 again once O8 passed.

O7 Post-copy

- Purpose: views, then procedures, counters and the stale check.
- Identity: runner; owner for the views and the procedures. A view or a procedure runs under its definer and C4 removes the runner, so the runner never creates one.
- Statements: USE, CREATE VIEW and CREATE PROCEDURE naming the replacement (owner), ALTER TABLE ... AUTO_INCREMENT and AGGREGATOR SYNC AUTO_INCREMENT ... ALL, counts and newest values on both sides.
- Source names: a view or procedure definition that names the source database refuses the step before any view statement. Edit the definition in the capture so it names no database, then rerun O7.
- Procedures: once every view exists, the owner session creates the captured procedures as O5 created the functions (rounds, the DEFINER rule, the 1227 fallback recorded per procedure), so a procedure that reads a view compiles and a pipeline INTO PROCEDURE at O9 finds its target. The gate compares each definition with the capture and records the ROUTINES row count of the replacement, which the switch expects on the live name after S3. Without the owner profile the procedures wait for the views, as the step ends pending.
- Views with the owner profile:

1. The owner session creates each missing view. It keeps the original DEFINER when that account still exists and is not the runner. Otherwise it removes the clause and the owner account becomes the definer. When the server refuses the original DEFINER (error 1227: Helios does not let even the admin name another account), the owner creates the view again without the clause. The journal and the Views tab of the final workbook record the choice per view.
2. A view that fails is tried again after the others, in rounds, so a view on another view is created once the other exists. Gate: every view created. A round that creates none refuses the step with each view and its error number; fix the cause and rerun O7. Views already created are skipped.

- Views without the owner profile:
  1. O7 prints the O7 block and writes it to `owner-sql/O7.sql`: every view in dependency order, each skipped when it exists and checked after. The admin who runs it becomes the definer.
  2. The stale check and the counters run. O7 ends pending.
  3. Run the block as the admin, then `s2rebuild step O7`. Gate: the rerun finds every view and passes.
- Counters: each AUTO_INCREMENT counter is set to the larger of the source counter and MAX + 1, then synced on every aggregator (ALL). A sharded table whose counter the server refuses to set (error 1749) gets the sync alone, which moves every aggregator past MAX. An empty table without a source counter keeps the default. The table tab of the final workbook shows both values and the decision. A rerun of O7 sets and syncs every counter again.
- Re-copy: a stale mismatch marks the table for re-copy, the kit copies the marked buckets again and repeats the check, for up to three rounds. A round that ends with every chunk done and the count proof all matching marks O6 passed again. The counters are set once, after the last round.
- Gate: views and procedures created, counters set, zero stale mismatches after re-copy.
- When it stops: a re-copy round that does not complete leaves O6 pending; run `s2rebuild run --from O6`, which finishes the marked buckets and reruns O7.

O8 Verify

- Purpose: prove the copy.
- Identity: runner.
- Tiers: 1 counts both sides; 2 newest time value and key range; 3 grouped fingerprints (key hash and value hash per group, mapped to buckets; the value hash takes the time column plus up to `verify.value_columns` supported columns).
- Tier 3 runs on `verify.sessions` connections (default `copy.workers`), largest table first, the source and replacement sides of a table at the same time; one log line per table with the elapsed seconds.
- A table with more rows than `verify.full_fingerprint_max_rows` (default 1,000,000,000) is sampled: a `verify.sample_percent` (default 5) range on its first sort key column or its time column, MIN and MAX from the source, the same predicate on both sides. A table over the threshold with neither column gets a full fingerprint with a note. The verification row and the table tab show the predicate and the rows in the sample.
- `verify.fingerprint_budget_minutes` (default none) is an elapsed deadline: a table whose turn comes after it is recorded not fingerprinted.
- Outputs: verification rows, each with the table's re-copy epoch and the sample predicate, `verified_at_utc` per table, the final workbook and run report.
- Gate: tiers 1 and 2 pass everywhere; tier 3 a full match on every table, or a sampled or not fingerprinted result accepted with `s2rebuild step O8 --yes`. With `--yes` and no chunk copied since the last O8

run, the recorded result is accepted without a rerun and an `accepted` event names each table; otherwise the tiers run again first.

- Rerun one table: `s2rebuild step O8 --fingerprint TABLE --percent N` (sampled) or `s2rebuild step O8 --full TABLE` runs tier 3 for the named tables only and records the result at the current epoch; the other tables keep their recorded result. Add `--redo` when O8 already passed.
- When it fails: exit 3; the mismatching buckets are marked for re-copy, which reopens O6 and O7. Run `s2rebuild run --from O6`: O6 copies them again, O7 resets the counters, O8 verifies again.

O9 Pipelines

- Purpose: every pipeline exists in the replacement, stopped, at the offsets recorded at O2. O9 runs after O7, so a pipeline INTO PROCEDURE finds the procedure O7 created; P3 and O4 refuse a load target whose procedure is ignored by configuration or absent from the capture.
- Identity: owner.
- Credentials: the server hides CREDENTIALS from every identity, so the customer supplies them. Route kit: the kit reads each pipeline's credentials from the Keychain item named in `pipelines.per_pipeline_keychain` or `pipelines.credentials_keychain` (filled at the P3 prompt), else from the variable named in `pipelines.per_pipeline_env` or `pipelines.credentials_env`, exported for this step only and unset afterwards; a source type still without credentials is asked for at O9 as at P3, and a blank answer skips it (its pipelines are left out of the count gate and listed in the block, the log and the final report). The kit injects the value as a SQL literal at execution and masks it in every log line. In the notebook the cell asks for each value with a hidden prompt. Route customer: the customer runs `pipelines/recreate-with-placeholders.sql` from the O2 capture inside the replacement, filling in CREDENTIALS; the kit runs only the checks.
- Statements: USE replacement, CREATE PIPELINE per pipeline (skipped when it exists), its ALTER PIPELINE ... SET OFFSETS statements. Never START.
- Gate: count equals the capture minus application-managed pipelines; none Running; the committed offset of every partition in the replacement equals the O2 capture. The committed offset is LATEST_SUCCESSFUL_OFFSET; O2 refuses a capture that holds a loaded offset without a successful one. The block checks each partition, so the notebook and hand routes hold the same gate.
- Refusals: only the O2 capture is replayed (an earlier capture's offsets were still moving); a definition that names the source database is refused. A definition that still carries a redaction marker (a value the server masked) is refused by name before any statement, by the kit and by the notebook cell; capture it again.

O10 Switch

- Purpose: the live name resolves to the replacement, the original stays readable, and the workspace ends with two entries: the live database and the retained copy.
- Identity: owner.
- S0 (decision): O8 and O9 passed; every planned table verified after freeze_time and after its last chunk, at its current re-copy epoch (a table without a stamp refuses, by name); both storage ids recorded; time left before the abort deadline; the operator types the live name (or passes `--yes`). The go is recorded only while the journal part of S0 still holds, and O10 holds the copy lock from S0 to S4 (O6 Copy).
- S0 on every route: without the owner profile O10 refuses on the journal part of S0 (O8 and every step before it passed, freeze_time, every table verified) and prints the forward block with a STOP first. `s2rebuild owner-sql O10` and the notebook O10 cell carry the same STOP while it holds, and `s2rebuild gate O10 --pass` refuses. Generate the forward block or the notebook after O8 passed,

and again after a re-copy (O6 Re-copy, step 4).

- A forward block generated before a re-copy: the kit rewrites the files under `owner-sql/` with a STOP, but cannot reach a notebook already uploaded. When `gate O10 --pass` refuses and its `--evidence` shows the switch ran (the retained and live attachments), the kit records those attachments and the pass stays unrecorded. From then on a re-copy is refused. The refusal names the rollback block. Its proof needs the copied row counts; when a re-copy cleared them, the block stops and the way back is a reconciliation. Without evidence, record what ran with `s2rebuild gate O10 --fail --note TEXT --evidence FILE`.
- When S0 refuses: the refusal names each table and the way to verify again. `s2rebuild step O8 --redo` when O7 passed after the last chunk; `s2rebuild run --from O6` while O7 is pending after a re-copy.
- S1: checks (source on its recorded STORAGE_ID and its recorded record, attach type for the freeze path, no foreign active session, replacement READWRITE as its recorded record, retained name free; on Path B every write privilege the lockout took still revoked, one check per account and scope, and the last lockout pass quiet within `freeze.quiet_max_age_seconds`), DETACH (with the wait and KILL rule), checks: the storage detached, the name free, and the original's detached record with its ATTACH_SYNTAX (the AT MILESTONE form), which the kit records for the rollback. The kit runs the before-checks once before the go and again before the DETACH. The Path B pass check reads the journal, so the printed block and the notebook carry it as a note; their privilege checks stop them before the DETACH when a grant came back after the quiet proof. Revoke it again as the O3 lockout plan does, kill that account's sessions, and run the block again from the top. Generate the block or the notebook after O3 recorded the snapshot; without one the block says it has no privilege check.
- S2: ATTACH source AS retained READ ONLY STORAGEID ..., checks (READONLY, a resume record of the original, recorded), one small table count.
- S2b (the live name is free): the original record's own ATTACH_SYNTAX (the AT MILESTONE form S1 recorded) attaches the original read write under the live name; checks: the recorded record, attached READWRITE; the zero-change proof: the three smallest non-empty tables hold the journal's source counts and the S1 session query shows no foreign non-idle session; then DROP DATABASE of the original record; checks: the record dropped, the name free, the original storage holds one entry (the retained record), one small table count in the retained copy. A failed proof stops with the original attached read write under the live name, the rollback state: reconcile, then `s2rebuild step O10` continues at the proof, or stay on the original. For those seconds the live name resolves to the original; the O1 decision that the application is stopped and the session check cover it.
- S3: checks, ATTACH replacement AS live STORAGEID ..., checks: READWRITE, not the original storage, a pitr record of the replacement (recorded), target layout, table, view, pipeline and routine counts (the routine count is the ROUTINES row count the O7 gate recorded), three smallest tables against the journal.
- S4: checks: the replacement name holds the recorded replacement record (create, the parent of the live record) and the live name is its own branch of it; DROP DATABASE of the replacement; checks: the record dropped, the name free, the live record unchanged, one small table count on the live name.
- Gate: the entries with IS_DROPPED = 0 on both storages are exactly the live record and the retained record (two entries). The listing before S1 and after S4 goes into the journal (`switch_listing`). A failed gate names the extra or missing entries; C3 carries the repair for a switch made without S2b and S4.
- Re-enter: the position comes from the attachment views. A rerun repeats the after-checks of the step it lands behind, so a failed check never turns into a passed switch.

- Write boundary: S3 attaches the live name READWRITE, so on Path A the application can write from S3, before O11.
- Rollback (before O11 only), by record: the zero-change proof when S3 ran, R1 detach the live name while it holds the recorded live record, then one of two legs. Before S2b ran: R2a, the original record's own ATTACH_SYNTAX (the AT MILESTONE form S1 recorded; the kit reads it from the live record and refuses when it differs from the recorded one); the retained copy stays attached read only. After S2b the original record is dropped, so the rollback goes through the retained copy: R2b, `ATTACH DATABASE <retained> AS <source> STORAGEID <original>`, a read-write pitr branch of the retained copy under the live name; when the server refuses a branch of a READ ONLY attachment, R2c: DETACH the retained name, then its record's own ATTACH_SYNTAX with AS <source>, so its record comes back read write under the live name; the journal and the report then say that the retained name is gone and that the data is under the live name. The live test I0b decides between R2b and R2c; the kit tries R2b and takes R2c on the refusal. No rollback step drops the retained copy. The rollback resumes from any position behind S1; behind S3 it starts at R2 without the proof. Every statement runs behind a record check: a record that differs refuses with both records printed and nothing issued. After a rollback through the retained copy the switch cannot continue; a new switch is a new freeze and copy.
- Zero-change proof: every planned table on the live name holds the row count of the copy, no pipeline is Running on it, no write privilege the lockout took is back, and no foreign non-idle session is active on it. It detects changed row counts and active writers only, not an update that keeps the count. After S3 a rollback is safe only when the O1 note records that the application stayed stopped.
- Rollback blocks: `s2rebuild owner-sql O10 --rollback` prints two, `O10-rollback.sql` (R1, R2a: before S2b ran) and `O10-rollback-retained.sql` (R1, R2b, then R2c skipped while the live name is attached: after S2b ran), and the notebook holds both. Use the one the S2b state calls for; the R2 before check of the wrong one stops it. Both carry the proof as checks before R1. Generate them or the notebook (`s2rebuild owner-notebook`) after O8 passed, so they hold the copied row counts. Generate them again just before a rollback, so they hold the O11 state and, after S1, the recorded ATTACH_SYNTAX of the original as the R2a statement; before S1 the block carries a milestone placeholder there and says so. The R2c re-attach needs the ATTACH_SYNTAX of the retained record after its DETACH; the block carries a placeholder and says where to copy it from.
- Behind S3 the notebook cell stops at the first proof check, with nothing issued. Finish by hand from the R2a or R2b check of the printed block.
- Rollback refused, before any statement: once O11 started releasing writers, once C1 saw application writes, or when the proof fails. The kit knows that O11 started from release_started_utc, released_utc or the O11 status. After a hand or notebook O11 the journal may not know, so the proof refuses on the live state: a pipeline Running or a privilege back. A failed proof lists what differs and says reconcile first: a new freeze and a reconciliation, never R1.
- Helios keeps the replacement's STORAGE_ID for the live branch. S3 then passes only when the live name has its own branch row (PREV_DATABASE_NAME the replacement, not dropped or detached) and a COMPUTE_ID other than the replacement's. A rerun of `s2rebuild step O10` at any position issues nothing it already did and reruns the after-checks of the step it lands behind. Identity everywhere is the branch record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), never a STORAGE_ID alone: the retained copy shares the original's STORAGE_ID and the live branch the replacement's.

O11 Release

- Purpose: writers back on the live name.
- Identity: owner and the customer.
- Retention check: for every captured partition the recorded committed offset must not be below the current EARLIEST_OFFSET, and the partition must exist. A gap means messages were lost to the source's retention; decide EARLIEST or LATEST per pipeline and record it in the gate note.
- Statements: GRANT restores (each checked) from any unrestored lockout snapshot, START PIPELINE per expected pipeline (skipped when Running), SET GLOBAL query_parallelism_per_leaf_core when configured.
- Release start: with the owner profile the kit records release_started_utc in the owner session, once the retention check and every check ahead of the first statement passed, just before that statement. From then on a rollback is refused, even when O11 stops part way. An O11 that stops before its first statement (a failed check, a lost connection) records nothing, and the rollback stays open. A hand or notebook O11 records nothing; there the rollback proof refuses once a pipeline runs or a privilege is back on the live name.
- Gate: every expected pipeline Running; grants equal the snapshot; the application restarted and writing. The kit stamps released_utc when every check passed; recording the gate by hand stamps it too.
- Re-enter: `s2rebuild step O11` after a failure. A pipeline already Running is skipped, so no START is issued twice.
- Pipelines left stopped: with `pipelines.start_at_release: none` O11 starts no pipeline. It checks that every captured partition is present and that none is Running, and notes the START PIPELINE form for later. C1 then reports the pipelines as left stopped and takes no offset sample.

C1 Verify after release

- Identity: runner; owner for pipelines and grants.
- Checks: writer watch, count changes on the largest tables (two samples a minute apart and against the copy), pipelines Running with offsets advancing where data waits, application-managed pipelines present again, no pipeline errors since the release, grants equal to the snapshot (Path B) or the reference (Path A). Application writes are recorded; from then on a rollback is a new freeze and a reconciliation.
- Gate: every check ok and application writes seen on the live database (a session, counts moving, or counts that differ from the copy). A live database that is idle on purpose passes with `s2rebuild step C1 --idle-workload NOTE`; the note is journaled. Without writes or the flag C1 fails with exit 4. A missing application-managed pipeline fails C1. Without the owner profile the pipeline and grants checks are recorded as not checked, no owner profile, and C1 passes with that note.
- Re-enter: `s2rebuild step C1`. It samples again; writes made earlier still show as counts that differ from the copy, and writes an earlier C1 attempt of the run saw (journal `app_writes_seen`) still count when the new sample sees none, with the time named in the note.
- When it fails: fix what the note names and rerun. When the kit cannot see the evidence, record C1 by hand with `s2rebuild gate C1 --pass --note TEXT --override NOTE`. Gate refuses C1 without `--override`; the override is journaled.

C2 Report and calibrate

- Identity: runner.
- Outputs: `reports/final.xlsx`, `reports/run-report.md`, `customer/baseline.json` refitted from the run's first-pass chunks when the journal can be fitted.
- Gate: reports written. Calibration is advisory: with too few chunks to fit (or a fit the model refuses) the baseline in use stays, the Summary of both reports carries a Calibration line that says so, and C2 passes with that note.
- Re-enter: `s2rebuild step C2` writes the reports and the baseline again from the same journal.
- When it fails: fix the cause and rerun. To record C2 by hand: `s2rebuild gate C2 --pass --note TEXT --override NOTE`. Gate refuses C2 without `--override`; the override is journaled.

C3 End state

- Identity: owner.
- Purpose: prove that the run left exactly two entries on the workspace, the live database and the retained copy, and record the listing.
- When: after C2. No waiting period: the retention period applies to C5, not to C3.
- Gate: the live name resolves to the recorded live record (a pitr record of the replacement) and holds no other record; the retained name holds the recorded retained record READONLY (a resume record of the original storage) and no other record; the entries with IS_DROPPED = 0 on the original storage and on the replacement storage are exactly those two; a write on the retained name is refused (the O3 write probe, run as the runner). The listing before and after goes into the journal as `endstate_listing` and into the final report.
- Statements: none on the kit route and in the printed block (reads and the write probe). The repair block is the exception below.
- Repair block, only for a switch made without S2b and S4 (an older kit, or the forward block of an older kit run by hand): the gate fails on a third entry (the detached original under the live name, the replacement name), and the journal shows no S2b or S4. The kit then prints o`wner-sql/C3-repair.sql` with the outage note (the live name is detached for about 30 seconds on a 3 TB database, Helios fact 34), asks "proceed?" unless `--yes`, and runs: detach the live name while it holds the recorded live record; the original record's own ATTACH_SYNTAX; its record and the three smallest table counts; DROP DATABASE of the original record; the live record's own ATTACH_SYNTAX (read live after the detach; the printed block carries a milestone placeholder and says where to copy it from); DROP DATABASE of the replacement by its record; the gate again. When the journal shows S2b and S4, a failed gate refuses with the listing and no repair: find out who attached the extra entry.
- Re-enter: `s2rebuild step C3`. The gate is reads only, so a rerun is free; a repair that stopped part way resumes from the live state (an original already dropped is not attached again, a replacement already gone is not dropped again).
- When it fails: the note names the check, the expected value and the observed value, and the listing of both storages. The retained copy attached READWRITE, or a record that differs from the recorded one, means someone moved a database; C3 never detaches, attaches or drops a database the run did not record.
- Then: keep the evidence folder. The retained copy stays until the customer deletes it (C5).

C4 Remove the migration account

- Identity: owner.
- Preconditions: C3 passed; no view and no routine in the live database has the runner as its definer.
- Statements: a check that the account still holds privileges, the definer check, DROP USER, a check that zero privileges remain.
- When it stops at the view check: a view whose definer is gone can no longer be executed. List them with `SELECT TABLE_NAME, DEFINER FROM information_schema.VIEWS WHERE TABLE_SCHEMA = '<live database>'`, recreate each one the runner defines as a durable account (the admin or the application account), then rerun `s2rebuild step C4`.
- Then: delete the Keychain item that held the runner password.

C5 Delete the retained original (optional, the customer)

- Command: `s2rebuild cleanup` lists what the run left after C3: the live record, the retained copy with its record (name, STORAGE_ID, ATTACH_TYPE, COMPUTE_ID), its recovery syntax, its size in GiB from the capture, the switch time, `retain_until_utc` and the C3 status. `s2rebuild cleanup --delete <retained>` deletes the retained copy.
- Identity: owner. The kit never runs it by itself; no step and no configuration key deletes the retained copy.
- Refused, with nothing issued and the reason journaled: a name other than the run's retained name; no retained record in the journal; no `retain_until_utc` (C2 records it); today before `retain_until_utc` ; C3 not passed; a record under the name that differs from the recorded one, or not READONLY.
- Two confirmations: the `--delete NAME` flag, then the name typed again at the prompt. Any other input keeps the copy, records "kept by customer" and exits 0. `--yes` does not apply; without a terminal the command refuses.
- Statements, each behind its record check (a READ ONLY attachment cannot be dropped, Helios fact 28): DETACH DATABASE retained; the detached record's own ATTACH_SYNTAX (read write); a check that the record is attached READWRITE; DROP DATABASE retained; checks that the record is dropped and the name free, and that the live name still resolves to the live record. The journal records `retained_deleted`.
- Then: the workspace holds the live database alone. The dropped record stays recoverable with `ATTACH DROPPED DATABASE` for the retention period of the engine.

Abandon before the switch

- Purpose: back out a run that stops before O10, for example to start again on a larger workspace.
- Identity: owner.
- Before: `s2rebuild pause`, then wait until the copy step ends. abandon takes the copy lock and refuses while a copy runs.
- Statements: checks that the source name holds the recorded record, that the replacement name holds the record O5 recorded or nothing, and that no branch was made from the replacement (a branch means the switch attached the live name from it); DETACH and ATTACH of the source READWRITE on its recorded STORAGE_ID, each behind a record check (only the ATTACH after a stop between them; checks only when the source is already READWRITE); DROP DATABASE of the replacement, skipped unless its name holds the recorded record; checks that the replacement record is dropped and the source is READWRITE as its recorded record.

- Commands: `s2rebuild abandon` prints the block; `s2rebuild abandon --execute --note TEXT` runs it. Gate: every check ok, and the log line "run ... abandoned".
- Then: `s2rebuild account-sql --drop --execute` removes the runner. `run`, `step` and `gate` refuse the abandoned run; a new run starts at P1.
- Refuses: once the switch may have begun (use `s2rebuild owner-sql O10 --rollback`), on Path B, and before P2 recorded the source STORAGE_ID.
- Re-enter: rerun the same command; it reads the attachment state and finishes what is left.